Optimal Solutions for the Moving Target Vehicle Routing Problem with Obstacles via Lazy Branch and Price

Anoop Bhat[1] and Geordan Gutow[2] and Surya Singh[3] and
Zhongqiang Ren[4] and Sivakumar Rathinam[5] and Howie Choset[1]

***Abstract*— The Moving Target Vehicle Routing Problem with Obstacles (MT-VRP-O) seeks trajectories for several agents that collectively intercept a set of moving targets. Each target has one or more time windows where it must be visited, and the agents must avoid static obstacles and satisfy speed and capacity constraints. We introduce Lazy Branch-and-Price with Relaxed Continuity (Lazy BPRC), which finds optimal solutions for the MT-VRP-O. Lazy BPRC applies the branch-and-price framework for VRPs, which alternates between a restricted master problem (RMP) and a pricing problem. The RMP aims to select a sequence of target-time window pairings (called a tour) for each agent to follow, from a limited subset of tours. The pricing problem adds tours to the limited subset. Conventionally, solving the RMP requires computing the cost for an agent to follow each tour in the limited subset. Computing these costs in the MT-VRP-O is computationally intensive, since it requires collision-free motion planning between moving targets. Lazy BPRC defers cost computations by solving the RMP using lower bounds on the costs of each tour, computed via motion planning with relaxed continuity constraints. We lazily evaluate the true costs of tours as-needed. We compute a tour's cost by searching for a shortest path on a Graph of Convex Sets (GCS), and we accelerate this search using our continuity relaxation method. We demonstrate that Lazy BPRC runs up to an order of magnitude faster than two ablations.**

I. INTRODUCTION

Finding trajectories for multiple agents to visit multiple moving targets is necessary in applications such as defense [1], [2], [3], orbital refueling [4], and recharging mobile robots collecting data from the seafloor [5]. These applications can be modeled as variations of the Vehicle Routing Problem (VRP) [6], [7]. The VRP assumes a set of stationary targets and a set of agents, where the agents start at a common location called the depot. Each target has a demand of goods, and each agent has a capacity on the amount of goods it can deliver. Given the travel cost between every pair of targets, and between the targets and the depot, the VRP seeks a sequence of targets for each agent with minimal sum of costs, such that the sum of demands of targets visited by an agent does not exceed the capacity. In the Moving Target VRP (MT-VRP) [8], the targets are moving, and we seek not only a sequence of targets for each agent, but a trajectory. Each target must be met in a particular time window(s), and the agents have a speed limit. Prior work on the MT-VRP assumes piecewise-linear target trajectories [8], and we make the same assumption. When the agents must avoid static obstacles, we have the MT-VRP with Obstacles (MT-VRP-O), shown in Fig. 1.

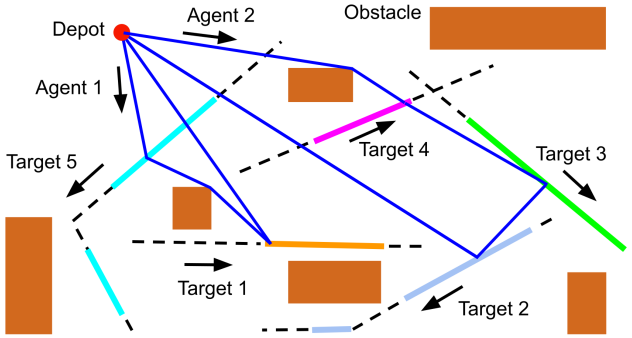

Fig. 1. Targets move through obstacle environment and must be intercepted within time windows, shown in bold-colored lines. Agents begin and end at depot, intercepting targets while avoiding obstacles.

The MT-VRP-O generalizes the Traveling Salesman Problem (TSP), and thus finding an optimal solution is NP-hard [9], [1]. No prior methods find an optimal solution for the MT-VRP-O. The closest related work finds optimal solutions for the MT-VRP without obstacles [8], using the branch-and-price framework [10]. In this work, we develop a new branch-and-price algorithm for the MT-VRP-O called Lazy Branch-and-Price with Relaxed Continuity (Lazy BPRC).

We define the pairing of a target with one of its time windows as a *target-window*. We define a tour as a sequence of target-windows, meant to be followed by a single agent. The cost of a tour is the distance traveled by a collision-free trajectory intercepting the tour's targets in order. The MT-VRP-O seeks a least-cost set of tours for the agents to follow, from the set of all possible tours. Since explicitly enumerating all possible tours is intractable, we employ *column generation* [11], where we initially generate a limited subset $\mathcal{F}$ of all possible tours, then alternate between selecting a set of tours from $\mathcal{F}$ and adding tours to $\mathcal{F}$.

Traditionally, the selection step within column generation (known as the restricted master problem) aims to minimize the sum of selected tours' costs. In the MT-VRP-O, however, computing tour costs is expensive, since it requires collision-free motion planning. Our key idea is to instead perform column generation using cheap-to-compute *lower bounds* on tour costs. We compute these lower bounds by solving a

[1]Robotics Institute at Carnegie Mellon University, 5000 Forbes Ave., Pittsburgh, PA 15213. Emails: {agbhat, choset}@andrew.cmu.edu.
[2]Mechanical and Aerospace Engineering at Michigan Technological University, Houghton, MI 49931. Email: gmgutow@mtu.edu
[3]Robotics and AI Institute, Cambridge, MA 02142. Email: ssingh@rai-inst.com
[4]UM-SJTU Joint Institute and Department of Automation at Shanghai Jiao Tong University, Shanghai, China. Email: zhongqiang.ren@sjtu.edu.cn
[5]Department of Mechanical Engineering and Department of Computer Science and Engineering at Texas A&M University, College Station, TX 77843. Email: srathinam@tamu.edu

motion planning problem with relaxed continuity constraints. Thus, we incorporate an outer alternation between (i) column generation using lower bounds on tour costs, and (ii) lazily evaluating only the costs of tours selected by column generation. We evaluate the cost of a tour by searching for a shortest path in a Graph of Convex Sets (GCS) [12]; we use our continuity relaxation strategy to provide a heuristic for the search. If column generation selects a set of tours whose costs have all been evaluated, we terminate the alternation between (i) and (ii). Our numerical results show that Lazy BPRC runs up to 46 times faster than a non-lazy ablation, and up to 26 times faster than an ablation using an existing obstacle-unaware heuristic [13].

II. RELATED WORK

While the MT-VRP-O has not been studied in prior work, several related problems have been studied. [5] studies a multi-agent Moving Target TSP with Obstacles (multi-agent MT-TSP-O), which lacks the capacity constraints from the MT-VRP-O. However, their approach only allows interception at sampled points along the targets' trajectories, and thus [5] does not provide optimal solutions. On the other hand, for the single-agent MT-TSP-O, [13] presents a solver that finds optimal solutions. [13] alternates between a high-level search to generate a tour, and a low-level search to find a trajectory intercepting the tour's targets to determine its cost. The low-level search in [13] solves a Shortest Path Problem on a GCS (SPP-GCS). We similarly solve an SPP-GCS to evaluate a tour's cost, and we provide a novel heuristic for the search that we show outperforms the heuristic from [13].

[8] studies the MT-VRP without obstacles using an approach called Branch-and-Price with Relaxed Continuity (BPRC). Our approach, Lazy BPRC, extends BPRC to handle obstacles, using a new obstacle-aware continuity relaxation strategy, as well as lazy tour cost evaluation. We show in Section VII that our lazy evaluation outperforms BPRC's non-lazy tour cost evaluation.

III. PROBLEM SETUP

We consider n_{tar} targets moving in $\mathbb{R}^2$, and $\{1, 2, \ldots, n_{\text{tar}}\}$ is the set of targets. Each target i has a demand d_i. Target i has $n_{\text{win}}(i)$ time windows, and $[\underline{t}_{i,j}, \overline{t}_{i,j}]$ is the jth time window of target i. The trajectory of target i is $\tau_i : \mathbb{R} \to \mathbb{R}^2$, and we assume τ_i has constant velocity within each time window, but possibly different velocities in different time windows. Without loss of generality, we assume targets do not pass through obstacles during their time windows.[1]

Let the number of agents be n_{agt}. Each agent has a capacity $d_{\max}$ on the amount of demand it can serve. When visiting a target, an agent must serve the target's full demand. Each agent has a speed limit $v_{\max}$, and no target moves faster than $v_{\max}$ within its time windows. We denote an agent's trajectory as τ_{a}. An agent trajectory τ_{a} *intercepts* target i if (i) $\tau_{\text{a}}(t) = \tau_i(t)$ for some t within some time window of target i, and (ii) τ_{a} *claims* target i at time t. The notion of claiming is needed when we plan a trajectory τ_{a} to intercept some target i, then i', but τ_{a} matches space-time locations with some target i'' unintentionally. As long as τ_{a} does not claim i'', the agent's capacity is not depleted when it meets i''. All agents start at a depot $p_{\text{d}} \in \mathbb{R}^2$. Finally, the agents must avoid collisions with stationary obstacles. We refer to a collision-free agent trajectory satisfying the speed limit as a *feasible* agent trajectory.

The MT-VRP-O seeks a feasible trajectory for each agent such that every target is intercepted by some agent's trajectory, and for each agent, the sum of demands of targets it intercepts does not exceed its capacity. In this work, we aim to minimize the sum of the agents' distances traveled.

[1] If a target enters an obstacle within a time window, we can replace the single window with two time windows: one that ends when the target enters the obstacle, and another that begins when the target exits the obstacle.

IV. INTEGER LINEAR PROGRAM (ILP) FOR MT-VRP-O

Lazy BPRC considers a *target-window graph* $\mathcal{G}_{\text{tw}} = (\mathcal{V}_{\text{tw}}, \mathcal{E}_{\text{tw}})$. Each node in $\mathcal{V}_{\text{tw}}$ is a pairing of a target i with one of its time windows, called a *target-window*. For example, $\gamma_{i,j} = (i, [\underline{t}_{i,j}, \overline{t}_{i,j}])$ denotes the jth target-window of target i. $\mathcal{V}_{\text{tw}}$ contains all possible target-windows, as well as a fictitious target-window $\gamma_{0,1} = \gamma_0 = (0, [0, \infty))$, referring to a fictitious stationary target 0 at the depot, with time window $[0, \infty)$. An agent trajectory τ_{a} *intercepts* target-window $\gamma_{i,j}$ if τ_{a} intercepts target i at some $t \in [\underline{t}_{i,j}, \overline{t}_{i,j}]$.

$\mathcal{E}_{\text{tw}}$ contains an edge from $\gamma_{i,j}$ to $\gamma_{i',j'}$ if $i \neq i'$. Each edge $(\gamma_{i,j}, \gamma_{i',j'}) \in \mathcal{E}_{\text{tw}}$ contains a value $\text{LFDT}(\gamma_{i,j}, \gamma_{i',j'}, \overline{t}_{i',j'})$, called the *latest feasible departure time*. The LFDT is the latest time $t \in [\underline{t}_{i,j}, \overline{t}_{i,j}]$ such that a feasible agent trajectory exists beginning at space-time point $(\tau_i(t), t)$ and intercepting $\gamma_{i',j'}$ at time $\overline{t}_{i',j'}$. We compute LFDT for all edges at the beginning of BPRC using the method from [14].

A *tour* is a path in $\mathcal{G}_{\text{tw}}$ beginning and ending at γ_0, visiting at most one target-window per non-fictitious target, such that (i) the sum of demands of visited targets is no larger than $d_{\max}$, and (ii) a feasible agent trajectory exists intercepting the target-windows in the tour in sequence. For a tour Γ, let $\Gamma[n]$ denote the nth element of Γ; in the subsequent text, we use the same bracket notation to indicate the nth element of any sequence. Let $\text{Len}(\Gamma)$ denote the number of target-windows in Γ. An agent trajectory τ_{a} *executes* Γ if τ_{a} intercepts the target-windows in Γ in sequence, and τ_{a} is feasible. For a tour Γ, the cost of Γ, denoted as $c^*(\Gamma)$, is the distance traveled by a minimum-distance trajectory executing Γ. We compute the cost of a tour by solving an SPP-GCS, described in Section V-G.

Let the set of all tours be $\mathcal{S}$. Lazy BPRC formulates the MT-VRP-O as the problem of selecting a set $\mathcal{F}_{\text{sol}} \subseteq \mathcal{S}$, containing up to n_{agt} tours, such that every target is visited by some selected tour, and the sum of tour costs is minimized. In particular, for a tour Γ, let $\alpha(i, \Gamma) = 1$ if Γ visits target i and let $\alpha(i, \Gamma) = 0$ otherwise. Define a binary variable θ_k which equals 1 if tour k is selected and 0 otherwise. We

formulate the MT-VRP-O as the following ILP:

$$\min_{\{\theta_k\}_{\Gamma_k \in \mathcal{S}}} \quad \sum_{\Gamma_k \in \mathcal{S}} c^*(\Gamma_k)\theta_k \tag{1a}$$

$$\text{s.t.} \quad \sum_{\Gamma_k \in \mathcal{S}} \theta_k \leq n_{\text{agt}} \tag{1b}$$

$$\sum_{\Gamma_k \in \mathcal{S}} \alpha(i, \Gamma_k)\theta_k \geq 1 \;\; \forall i \in \{1, \ldots, n_{\text{tar}}\} \tag{1c}$$

$$\theta_k \in \{0, 1\} \;\; \forall k \in \{1, \ldots, |\mathcal{S}|\} \tag{1d}$$

(1a) minimizes the sum of tour costs, (1b) ensures that no more than n_{agt} tours are selected, (1c) ensures all targets are visited, and (1d) enforces that each θ_k is binary.

V. Lazy BPRC

A. Preliminaries

When solving ILP (1), explicitly having decision variables for every $\Gamma_k \in \mathcal{S}$ is intractable, since the number of tours grows factorially with the numbers of targets. Thus, Lazy BPRC maintains a subset $\mathcal{F} \subseteq \mathcal{S}$, which is enlarged throughout the algorithm, and only selects tours from $\mathcal{F}$. For each tour $\Gamma \in \mathcal{F}$, we maintain a lower bound $\underline{c}(\Gamma)$ and an upper bound $\overline{c}(\Gamma)$ on $c^*(\Gamma)$. At the time when we add a tour Γ into $\mathcal{F}$, we compute the lower bound using the method from Section V-D and the upper bound using the method from Section V-E, and we refer to Γ as *unevaluated*. Over the course of the algorithm, we compute $c^*(\Gamma)$ for certain tours Γ, then set their lower and upper bounds equal to $c^*(\Gamma)$; we refer to such tours Γ as *evaluated*. For a set of tours $\mathcal{F}_{\text{sol}}$ that is feasible for ILP (1), let $\overline{c}(\mathcal{F}_{\text{sol}}) = \sum_{\Gamma \in \mathcal{F}_{\text{sol}}} \overline{c}(\Gamma)$.

Algorithm 1 Lazy BPRC

```
1: F_inc = GenerateFeasibleSolution()
2: if F_inc = ∅ then return INFEASIBLE
3: F = Copy(F_inc)
4: STACK = [∅]
5: while STACK is not empty do
6:     B = STACK.pop()
7:     while true do
8:         θ, c_lb(θ) = SolveLP-B(F, F_inc)
9:         if θ is purely integer AND c_lb(θ) < c_ub_inc then
10:            F_sol = ExtractTours(θ, F)
11:            F_uneval = GetUnevaluatedTours(F_sol)
12:            ComputeTourCosts(F_uneval)
13:            if c_ub(F_sol) < c_ub_inc then F_inc = F_sol
14:        else break
15:    if c_lb(θ) ≥ c_ub_inc then continue
16:    B', B'' = GenerateSuccessors(B, θ, F)
17:    STACK.push(B')
18:    STACK.push(B'')
19: return F_inc
```

B. Branch and Bound

Lazy BPRC solves ILP (1) via the branch-and-bound procedure shown in Alg. 1. The algorithm begins by generating an initial feasible solution $\mathcal{F}_{\text{inc}}$ for ILP (1), using the method in Section V-H. We call $\mathcal{F}_{\text{inc}}$ the incumbent, and we continually update $\mathcal{F}_{\text{inc}}$ to be the best solution to ILP (1) found so far, where "best" refers to smallest $\overline{c}$-value. We initialize the subset $\mathcal{F}$, introduced in Section V-A, to $\mathcal{F}_{\text{inc}}$. Define $\overline{c}_{\text{inc}}$ as always taking the value of $\overline{c}(\mathcal{F}_{\text{inc}})$.

Next, we initialize a stack of *branch-and-bound nodes*, where each node $\mathcal{B}$ is a set of disallowed edges in $\mathcal{E}_{\text{tw}}$. Let ILP-$\mathcal{B}$ be ILP (1), with the constraint $\theta_k = 0$ for any Γ_k traversing an edge in $\mathcal{B}$. Let LP-$\mathcal{B}$ be the convex relaxation of ILP-$\mathcal{B}$ which replaces constraint (1d) with $\theta_k \geq 0$: LP-$\mathcal{B}$ is often called the *master problem* in branch-and-price. Let $c^*(\mathcal{B})$ be the optimal cost of LP-$\mathcal{B}$.

When we expand $\mathcal{B}$, we compute a lower bound on the optimal cost of ILP-$\mathcal{B}$, which we obtain from a lower bound on the optimal cost of LP-$\mathcal{B}$. In particular, we enter a loop that solves LP-$\mathcal{B}$ with lazy evaluation of tours in $\mathcal{F}$ (Line 7). On Line 8, we solve LP-$\mathcal{B}$, but replace $c^*(\Gamma_k)$ in the objective (1a) with $\underline{c}(\Gamma_k)$: we call this problem the *surrogate master problem*, LP-$\mathcal{B}$. We obtain a solution θ to LP-$\mathcal{B}$ using column generation, which may add tours to $\mathcal{F}$ and update $\mathcal{F}_{\text{inc}}$ (Section V-C). On Line 8, $\underline{c}(\theta)$ denotes the cost of θ within LP-$\mathcal{B}$. Note that θ may not be optimal for LP-$\mathcal{B}$, but we ensure that

$$\underline{c}(\theta) \leq c^*(\mathcal{B}) \tag{2}$$

as described in Section V-F.

We then check if θ is purely integer and $\underline{c}(\theta) < \overline{c}_{\text{inc}}$ (Line 9). If so, θ corresponds to a set of tours $\mathcal{F}_{\text{sol}}$ whose actual cost may be lower than $\overline{c}_{\text{inc}}$. Let $\mathcal{F}_{\text{uneval}}$ be the set of unevaluated tours in $\mathcal{F}_{\text{sol}}$. We must have $\mathcal{F}_{\text{uneval}} \neq \emptyset$, since as we explain in Section V-C, while solving LP-$\mathcal{B}$, whenever we obtain an integer solution θ whose corresponding set $\mathcal{F}_{\text{sol}}$ has all tours evaluated, we set $\mathcal{F}_{\text{inc}} = \mathcal{F}_{\text{sol}}$. Thus, we evaluate each $\Gamma_k \in \mathcal{F}_{\text{uneval}}$ by solving an SPP-GCS (Section V-G), then solve LP-$\mathcal{B}$ again.

After exiting the lazy evaluation loop, if $\underline{c}(\theta) \geq \overline{c}_{\text{inc}}$, we continue to the next expansion. Otherwise, the failure of the conditions on Lines 9 and 15 imply that θ contains non-integer values. We create two successors for $\mathcal{B}$, denoted as $\mathcal{B}'$ and $\mathcal{B}''$, such that θ is feasible for neither LP-$\mathcal{B}'$ nor LP-$\mathcal{B}''$, but all integer solutions to ILP-$\mathcal{B}$ are feasible for both ILP-$\mathcal{B}'$ and ILP-$\mathcal{B}''$. To do so, we apply "conventional branching" [15]. In particular, for an edge $e \in \mathcal{E}_{\text{tw}}$, define the *flow* along e as the sum of θ_k values for all Γ_k traversing e. We select the edge e with flow closest to 0.5 and let $\mathcal{B}' = \mathcal{B} \cup \{e\}$. We then define $\mathcal{B}''$ so that e is required be traversed by some tour in a solution to ILP-$\mathcal{B}''$, by disallowing other edges appropriately (see [15]). We push $\mathcal{B}'$ and $\mathcal{B}''$ onto the stack.

C. Column Generation

We now describe how we find a solution θ for LP-$\mathcal{B}$ satisfying (2). As stated in Section V-A, enumerating all the decision variables for LP-$\mathcal{B}$ is intractable, so we use column generation [11]. In particular, define the *restricted surrogate master problem* (RSMP) on $\mathcal{F}$ as LP-$\mathcal{B}$ with the constraint that $\theta_k = 0$ for all tours $\Gamma_k \notin \mathcal{F}$. Note that to solve the RSMP, we do not have to solve for θ_k with $\Gamma_k \notin \mathcal{F}$. To find

a solution θ to LP-$\mathcal{B}$ satisfying (2), we alternate between solving the RSMP on $\mathcal{F}$ and adding tours to $\mathcal{F}$. To find tours to add to $\mathcal{F}$, we solve a *pricing problem* (Section V-F). Whenever we obtain an integer solution θ to the RSMP, this corresponds to a feasible solution $\mathcal{F}_{\text{sol}}$ to the MT-VRP-O. In this case, if $\bar{c}(\mathcal{F}_{\text{sol}}) < \bar{c}_{\text{inc}}$, we set $\mathcal{F}_{\text{inc}} = \mathcal{F}_{\text{sol}}$.

We alternate between the RSMP and pricing problem until the pricing problem finds no new tours, and we return the optimal RSMP solution θ. The first time we solve the RSMP, it may be infeasible, because all feasible MT-VRP-O solutions that can be constructed from tours in $\mathcal{F}$ traverse some edge in $\mathcal{B}$. In this case, we use the method from Section V-H to generate a set of tours $\mathcal{F}_{\text{new}}$ feasible for the MT-VRP-O, add these tours to $\mathcal{F}_{\text{sol}}$, and solve the RSMP again. If we fail to generate $\mathcal{F}_{\text{new}}$, we return $\theta = \text{NULL}$ and $\underline{c}(\theta) = \infty$.

D. Computing Lower Bound on Tour Cost

This section describes how we compute the lower bound $\underline{c}(\Gamma)$ for a tour Γ in Section V. Our method, illustrated in Fig. 2, extends the procedure from BPRC [8] to handle obstacles. At the beginning of Lazy BPRC, we divide each target-window $\gamma_{i,j}$ into *segments*, where $\xi_{i,j,k} = (i, [\underline{t}_{i,j,k}, \bar{t}_{i,j,k}])$ denotes the kth segment of $\gamma_{i,j}$, and $\text{Segments}(\gamma_{i,j})$ is the set of segments of $\gamma_{i,j}$. To determine the number of segments per target-window, we first specify a number of segments to allocate per target, denoted as $n_{\text{seg,tar}}$. Then for each target i, we allocate segments to its windows using the formula from BPRC [8], which gives more segments to longer windows. The depot gets a single segment ξ_0. For a target-window γ, $n_{\text{seg}}(\gamma)$ is the number of segments allocated to γ. The segment indices for a target-window are ordered in increasing order of start time. An agent trajectory τ_{a} *intercepts* segment $\xi_{i,j,k}$ if τ_{a} intercepts target i at a time $t \in [\underline{t}_{i,j,k}, \bar{t}_{i,j,k}]$.

For every pair of segments (ξ, ξ') corresponding to different targets, we compute a lower bound $c_{\text{seg}}(\xi, \xi')$ on the cost of a feasible agent trajectory that intercepts ξ, then ξ'. To do so, we compute the shortest collision-free path in space from ξ to ξ', ignoring time constraints, via the method from [16]. Let c be the path's distance traveled. Let t be the start time of ξ, and let t' be the end time of ξ'. We set $c_{\text{seg}}(\xi, \xi') = c$ if $t + c/v_{\max} \le t'$, and $c_{\text{seg}}(\xi, \xi') = \infty$ otherwise.

Next, given a tour Γ, we define a *segment-graph* $\mathcal{G}_{\text{seg}} = (\mathcal{V}_{\text{seg}}, \mathcal{E}_{\text{seg}})$, where the set of nodes $\mathcal{V}_{\text{seg}}$ is the set of all segments whose target-windows are visited by Γ. For each edge $(\Gamma[n], \Gamma[n+1]) \in \mathcal{E}_{\text{tw}}$ traversed by Γ, we connect edges in $\mathcal{E}_{\text{seg}}$ from every segment of $\Gamma[n]$ to every segment of $\Gamma[n+1]$. The cost of each edge (ξ, ξ') is $c_{\text{seg}}(\xi, \xi')$.

Let $g_{\text{seg}}(\xi)$ be the cost of the shortest path in $\mathcal{G}_{\text{seg}}$ from ξ_0 to ξ. $\underline{c}(\Gamma)$, computed as follows, lower-bounds $c^*(\Gamma)$:

$$\underline{c}(\Gamma) = \min_{\xi \in \text{Segments}(\Gamma[\text{Len}(\Gamma)-1])} (g_{\text{seg}}(\xi) + c_{\text{seg}}(\xi, \xi_0)) \quad (3)$$

$g_{\text{seg}}(\xi_0) = 0$, and for a segment ξ' of $\Gamma[n]$ with $1 < n < \text{Len}(\Gamma)$, we have $g_{\text{seg}}(\xi') = \min_{\xi \in \text{Segments}(\Gamma[n-1])} (g_{\text{seg}}(\xi) + c_{\text{seg}}(\xi, \xi'))$. Thus to compute $\underline{c}(\Gamma)$ we can iterate from $n = 2$ to $n = \text{Len}(\Gamma) - 1$, and for each n, compute the g-values of the segments of $\Gamma[n]$ using the g-values for $\Gamma[n-1]$.

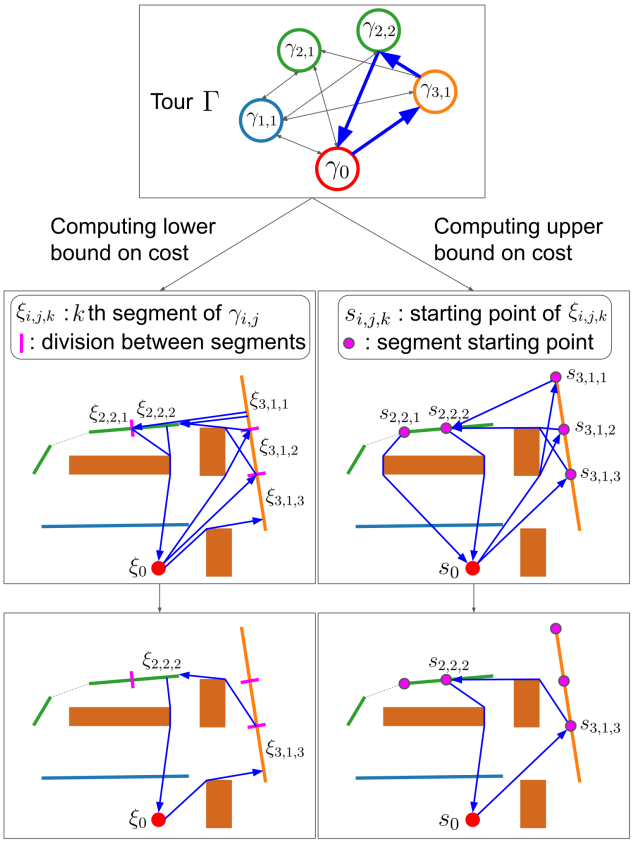

Fig. 2. Computing bounds on the cost of an example tour Γ. To compute the lower bound $\underline{c}(\Gamma)$, we divide each target-window visited by Γ into segments. We then construct a *segment-graph* $\mathcal{G}_{\text{seg}}$, where the nodes are the segments, and an edge connects every segment of $\Gamma[n]$ to every segment of $\Gamma[n+1]$. The edge cost from segment ξ to ξ' is the distance traveled along the shortest path in space from ξ to ξ', if this path satisfies the relaxed timing constraints from in Section V-D, and ∞ otherwise. $\underline{c}(\Gamma)$ is the cost of the shortest path in $\mathcal{G}_{\text{seg}}$ from ξ_0 to ξ_0 visiting all target-windows in Γ. To compute the upper bound $\bar{c}$, we construct a *segment-start-graph* $\mathcal{G}_{\text{start}}$, where the nodes are the starting points of the segments, and an edge connects every segment-start of $\Gamma[n]$ to every segment-start of $\Gamma[n+1]$. The edge cost from s to s' is distance traveled by a feasible minimum-distance agent trajectory from s to s', if such a trajectory exists or if $s' = s_0$, and ∞ otherwise. Our upper bound is the cost of the shortest path in $\mathcal{G}_{\text{start}}$ from s_0 to s_0 that visits all target-windows in Γ.

After this, we can compute $\underline{c}(\Gamma)$ via (3). As shown in Fig. 2, these computations correspond to finding an agent trajectory executing Γ subject to relaxed continuity constraints.

For tours generated in the pricing problem (Section V-F), these g-values are computed as a byproduct of solving the pricing problem. For tours generated using the feasible solution generation method in Section V-H, we perform these g-value computations after generating the tours.

E. Computing Upper Bound on Tour Cost

This section describes how we compute the upper bound $\bar{c}(\Gamma)$ for a tour Γ in Section V. Recall that in Section V-D, we divided each target-window into segments. Let the starting point in space-time of segment $\xi_{i,j,k}$ be $s_{i,j,k} = (\tau_i(\underline{t}_{i,j,k}), \underline{t}_{i,j,k})$. Denote the starting point of the depot segment as $s_0 = s_{0,1,1}$. For a target-window γ, let $\text{SegmentStarts}(\gamma)$ denote the set of segment-starts of γ. We construct a *segment-start-graph* $\mathcal{G}_{\text{start}} = (\mathcal{V}_{\text{start}}, \mathcal{E}_{\text{start}})$. The set

of nodes $\mathcal{V}_{\text{start}}$ is the set of all segment-starts whose target-windows are visited by Γ. For each edge $(\gamma, \gamma') \in \mathcal{E}_{\text{tw}}$ traversed by Γ, we connect an edge in $\mathcal{E}_{\text{start}}$ from every segment-start of γ to every segment-start of γ'.

To determine the cost of an edge from $s = (q, t)$ to $s' = (q', t')$, denoted as $c_{\text{start}}(s, s')$, we compute the shortest collision-free path in space from q to q' using [17]. Let the distance traveled by this path be c. If $t + c/v_{\max} \leq t'$ or $s' = s_0$, we set $c_{\text{start}}(s, s') = c$, and otherwise $c_{\text{start}}(s, s') = \infty$.

Define $g_{\text{start}}(s)$ as the cost of a shortest path in $\mathcal{G}_{\text{start}}$ from s_0 to s. $\overline{c}(\Gamma)$, computed as follows, upper-bounds $c^*(\Gamma)$:

$$\overline{c}(\Gamma) = \min_{s \in \text{SegmentStarts}(\Gamma[\text{Len}(\Gamma)-1])} (g_{\text{start}}(s) + c_{\text{start}}(s, s_0)) \quad (4)$$

To compute $\overline{c}(\Gamma)$, we note that $g(s_0) = 0$, and for a segment ξ' of $\Gamma[n]$ with $1 < n < \text{Len}(\Gamma)$, we have $g_{\text{start}}(s') = \min_{s \in \text{SegmentStarts}(\Gamma[n-1])} (g_{\text{start}}(s) + c_{\text{start}}(s, s'))$. Thus, we can iterate from $n = 2$ to $n = \text{Len}(\Gamma) - 1$, and for each n, compute the g-values of the segment-starts of $\Gamma[n]$ using the g-values for $\Gamma[n-1]$. Then we can compute $\overline{c}(\Gamma)$ using (4). For tours generated within the pricing problem (Section V-F), this computation happens as a byproduct and does not require extra computation. For tours generated using the feasible solution generation method in Section V-H, we must compute these g-values separately from the tour generation.

We place points $s_{i,j,k}$ at the segment-starts rather than at arbitrary points because, within the pricing problem (Section V-F), we need an upper bound on the cost of reaching each segment-start for dominance checking.

F. Pricing Problem

The pricing problem seeks a set of tours $\mathcal{F}_{\text{price}}$ such that the RSMP on $\mathcal{F} \cup \mathcal{F}_{\text{price}}$ has a smaller optimal cost than the RSMP on $\mathcal{F}$. To solve the pricing problem, we first note that when the RSMP produces a solution θ (specifically, its primal solution), the RSMP also produces a dual solution $(\lambda_0, \lambda_1, \ldots, \lambda_{n_{\text{tar}}})$, where $\lambda_0 \in \mathbb{R}_{\leq 0}$ is the dual variable corresponding to (1b), and for $i > 0$, $\lambda_i \in \mathbb{R}_{\geq 0}$ is the dual variable corresponding to (1c). Similarly to prior VRP work [11], we define the *reduced cost* of Γ as follows:

$$\underline{c}_{\text{red}}(\Gamma) = \underline{c}(\Gamma) - c_{\text{dual}}(\Gamma) \quad (5)$$

where $c_{\text{dual}}(\Gamma) = \sum_{i=1}^{n_{\text{tar}}} \alpha(i, \Gamma)\lambda_i + \lambda_0$. To improve the RSMP's optimal cost, $\mathcal{F}_{\text{price}}$ must contain a tour with negative reduced cost [11]. Therefore, as in prior work, we search for tours with negative reduced cost.

In contrast to prior work, we do not guarantee returning a tour with negative reduced cost if such a tour exists. Instead, we only guarantee returning some Γ such that $c^*(\Gamma) - c_{\text{dual}}(\Gamma) < 0$, if such Γ exists. This is sufficient to ensure that if we do not return any tours, then (2) is satisfied, as shown in the proof of Lemma 1.

We search for tours with negative reduced cost by modifying the labeling algorithm from [8] to handle obstacles. First, we define a *partial tour*, which has the same definition as a tour from Section IV, except that a partial tour does not need to end with the depot.

1) Labels and Label Dominance: Within our labeling algorithm, we represent a partial tour Γ using a *label* $l = (\gamma_{i,j}, t, \sigma, \vec{b}, \vec{g}_{\text{ub}}, \vec{g}_{\text{lb}}, \lambda)$, where

- $\gamma_{i,j}$ is the final target-window in Γ
- t is the minimum time required to execute Γ
- σ is the sum of demands of targets visited by Γ
- $\vec{b}$ is a binary vector with length n_{tar} where $\vec{b}[i'] = 1$ for any target i' such that either (i) Γ visits i', (ii) $t >$ LFDT$(\gamma_{i,j}, \gamma_{i',j'})$ for all $j' \in \{1, 2, \ldots, n_{\text{win}}(i')\}$, or (iii) $\sigma + d_{i'} > d_{\max}$
- $\vec{g}_{\text{ub}}$ is a vector with length $n_{\text{seg}}(\gamma_{i,j})$ where, if Γ is not a tour, $\vec{g}_{\text{ub}}[k] = g_{\text{start}}(s_{i,j,k})$ within the segment-start-graph for all tours extending from Γ. If Γ is a tour, $\vec{g}_{\text{ub}}$ contains a single element equal to $\overline{c}(\Gamma)$, as computed in Section V-E (i.e. not the true tour cost)
- $\vec{g}_{\text{lb}}$ is a vector with length $n_{\text{seg}}(\gamma_{i,j})$ where, if Γ is not a tour, we have $\vec{g}_{\text{lb}}[k] = g_{\text{seg}}(\xi_{i,j,k})$ within the segment-graph for all tours extending from Γ. If Γ is a tour, $\vec{g}_{\text{lb}}$ contains a single element equal to $\underline{c}(\Gamma)$, as computed in Section V-E (i.e. not the true tour cost)
- $\lambda = c_{\text{dual}}(\Gamma)$

Consider two labels $l = (\gamma_{i,j}, t, \sigma, \vec{b}, \vec{g}_{\text{ub}}, \vec{g}_{\text{lb}}, \lambda)$ and $l' = (\gamma_{i,j}, t', \sigma', \vec{b}', \vec{g}'_{\text{ub}}, \vec{g}'_{\text{lb}}, \lambda')$, both at target-window $\gamma_{i,j}$. If $\gamma_{i,j} = \gamma_0$, we say l *dominates* l' if $\vec{g}_{\text{ub}}[1] - \lambda \leq \vec{g}'_{\text{lb}}[1] - \lambda$. Otherwise, l dominates l' if

$$\sigma \leq \sigma' \quad (6)$$

$$\vec{b}[i'] \leq \vec{b}'[i'], \quad \forall i' \in \{1, \ldots, n_{\text{tar}}\} \quad (7)$$

$$\vec{g}_{\text{ub}}[k] + \delta(\xi_{i,j,k}) - \lambda \leq \vec{g}'_{\text{lb}}[k] - \lambda', \ \forall k \in \{1, \ldots, n_{\text{seg}}(\gamma_{i,j})\} \quad (8)$$

where $\delta(\xi_{i,j,k})$ is the length of segment $\xi_{i,j,k}$ in space. Let Γ be the partial tour represented by l and Γ' be the partial tour represented by l'. Let Ω be the tour extending Γ such that $c^*(\Omega) - c_{\text{dual}}(\Omega)$ is minimal, and let Ω' be the tour extending Γ' such that $c^*(\Omega') - c_{\text{dual}}(\Omega')$ is minimal. If l dominates l' by our definition above, then $c^*(\Omega) - c_{\text{dual}}(\Omega) < c^*(\Omega') - c_{\text{dual}}(\Omega')$ ([8], Theorem 1).

In particular, (6) and (7) are standard dominance conditions from the VRP literature [10], ensuring that any sequence of target-windows that can be appended to Γ can also be appended to Γ' without the violating the capacity constraint or causing a target to be revisited.

Condition (8) is specific to moving targets. Consider trajectories τ_{a} and $\tau_{\text{a}}{}'$ that execute Γ and Γ', respectively, and terminate by intercepting $\xi_{i,j,k}$, with minimum distance traveled. Define the reduced cost of τ_{a} as the distance traveled minus λ, and the reduced cost of $\tau_{\text{a}}{}'$ likewise using λ'. The term $\vec{g}_{\text{ub}}[k]$ on the LHS of (8) upper-bounds the distance an agent must travel to execute Γ and travel to the start of $\xi_{i,j,k}$, and $\delta(\xi_{i,j,k})$ is the distance an agent must travel from the start of $\xi_{i,j,k}$ to the end, i.e. to visit every point in the segment. Thus, the sum of these two terms upper-bounds the cost of τ_{a}, since τ_{a} cannot do worse than travel to the start of $\xi_{i,j,k}$, then move along $\xi_{i,j,k}$ to the point of interception. Thus the LHS upper-bounds the reduced cost of τ_{a}. The $\vec{g}'_{\text{lb}}$ lower-bounds the distance traveled by $\tau_{\text{a}}{}'$, so the RHS lower-bounds the reduced cost of $\tau_{\text{a}}{}'$. If condition

(8) holds, τ_{a} cannot be worse (i.e. cannot have more positive reduced cost) than $\tau_{\text{a}}{}'$ and we can discard l'.

2) Labeling Algorithm: Our labeling algorithm maintains a set of mutually nondominated labels at each target-window, as well as a priority queue, where labels with lexicographically smaller $(t, \sigma, \min(g_{\text{lb}}) - \lambda)$ have higher priority. When we expand a label $l = (\gamma_{i,j}, t, \sigma, \vec{b}, \vec{g}_{\text{ub}}, \vec{g}_{\text{lb}}, \lambda)$, we iterate over all *successor target-windows* of l, i.e. all target-windows $\gamma_{i',j'}$ satisfying the following conditions:

1) $(\gamma_{i,j}, \gamma_{i',j'}) \in \mathcal{E}_{\text{tw}} \setminus \mathcal{B}$
2) $t \leq \text{LFDT}(\gamma_{i,j}, \gamma_{i',j'})$
3) $\sigma + d_{i'} \leq d_{\max}$
4) If $i' \neq 0$, then $\vec{b}[i'] = 0$

For each successor target-window $\gamma_{i',j'}$, we generate a successor label $l' = (\gamma_{i',j'}, t', \sigma', \vec{b}', \vec{g}'_{\text{ub}}, \vec{g}'_{\text{lb}}, \lambda')$, where

- $t' = \text{EFAT}(\gamma_{i,j}, \gamma_{i',j'}, t)$, where EFAT is the earliest time at which a feasible agent trajectory can intercept $\gamma_{i',j'}$ after intercepting $\gamma_{i,j}$ at time t. EFAT stands for "earliest feasible arrival time," and we compute it using the method from [14].
- $\sigma' = \sigma + d_{i'}$
- $\vec{b}'$ is identical to $\vec{b}$, except for the following modifications. First, if $i' \neq 0$, we set $\vec{b}'[i'] = 1$. Then, we set $\vec{b}'[i''] = 1$ for each target i'' such that either (i) $\sigma' + d_{i''} > d_{\max}$, or (ii) for all $j'' \in \{1, 2, \ldots, n_{\text{win}}(i'')\}$, $t' > \text{LFDT}(\gamma_{i',j'}, \gamma_{i''j''})$.
- For each $k' \in \{1, 2, \ldots, n_{\text{seg}}(\gamma_{i',j'})\}$, we compute

$$\vec{g}'_{\text{ub}}[k'] = \min_{k \in \{1,2,\ldots,n_{\text{seg}}(\gamma_{i,j})\}} \vec{g}_{\text{ub}}[k] + c_{\text{start}}(s_{i,j,k}, s_{i',j',k'}) \quad (9)$$

- For each $k' \in \{1, 2, \ldots, n_{\text{seg}}(\gamma_{i',j'})\}$, we compute

$$\vec{g}'_{\text{lb}}[k'] = \min_{k \in \{1,2,\ldots,n_{\text{seg}}(\gamma_{i,j})\}} \vec{g}_{\text{lb}}[k] + c_{\text{seg}}(\xi_{i,j,k}, \xi_{i',j',k'}) \quad (10)$$

- $\lambda' = \lambda$, if $i' = 0$, and $\lambda' = \lambda + \lambda_{i'}$ otherwise

We then check if any labels at $\gamma_{i',j'}$ dominate l'. If so, we prune l'. Otherwise, we prune labels at $\gamma_{i',j'}$ dominated by l', and we mark them to be discarded upon expansion from the priority queue. Then we push l' onto the priority queue. Any time we generate a successor label l at γ_0 with $\vec{g}_{\text{lb}}[1] - \lambda < 0$,[2], and l is not dominated, we reconstruct the tour Γ represented by l via backpointer traversal, then add Γ to the set of tours to be returned. At this step, if Γ has already been evaluated, we do not add it to this set, since $c^*_{\text{red}}(\Gamma)$ cannot be negative. The search ends when the priority queue becomes empty.

G. Computing Tour Cost via SPP-GCS

At the beginning of Lazy BPRC, we decompose free space into convex regions $\mathcal{A}_1, \mathcal{A}_2, \ldots, \mathcal{A}_{n_{\text{reg}}}$, where n_{reg} is the number of regions. We then define a GCS $\mathcal{G}_{\text{cs}} = (\mathcal{V}_{\text{cs}}, \mathcal{E}_{\text{cs}})$, where the set of nodes $\mathcal{V}_{\text{cs}}$ consists of convex sets in space-time. For each region $\mathcal{A}$ in our free space decomposition, we have a *region-node* $\mathcal{X}_{\mathcal{A}} = \mathcal{A} \times \mathbb{R}$. For each target-window $\gamma_{i,j}$ visited by Γ, we have a *window-node* $\mathcal{X}_{i,j}$, consisting of the set of space-time points along τ_i within $[\underline{t}_{i,j}, \bar{t}_{i,j}]$. Since we assumed that a target's velocity is constant within a time window, $\mathcal{X}_{i,j}$ is a line segment in space-time, which is a convex set. We refer to nodes in $\mathcal{G}_{\text{cs}}$ as *GCS-nodes*. An edge connects a set $\mathcal{X}$ to a set $\mathcal{X}'$ if $\mathcal{X}$ intersects $\mathcal{X}'$. We refer to a path in $\mathcal{G}_{\text{cs}}$ as a *GCS-path*. We say a GCS-path P *visits* target-window $\gamma_{i,j}$ if P contains $\mathcal{X}_{i,j}$.

We find a trajectory executing Γ using an algorithm similar to FMC* [13], but without the various speedup techniques that FMC* implements particularly for a minimum-time objective, since we aim to minimize distance traveled. We also replace the heuristic from FMC* with the heuristic described later in the section, and we replace the focal search from FMC* with a best-first search, since we seek optimal solutions rather than bounded-suboptimal solutions.

We search the GCS using a priority queue called OPEN, containing GCS-paths. We initialize OPEN with the GCS-path $(\mathcal{X}_{0,1})$, i.e. a path that stays at the depot. Each GCS-path on OPEN has an f-value. The initial GCS-path $(\mathcal{X}_{0,1})$ has an f-value of 0; we discuss the computation of f for other GCS-paths shortly. GCS-paths with smaller f-values have higher priority.

Each iteration pops a GCS-path P from OPEN, then iterates over each GCS-node $\mathcal{X}$ adjacent to $P[-1]$. If $\mathcal{X}$ is a window-node $\mathcal{X}_{i,j}$, and P has not visited the target-windows occurring before $\gamma_{i,j}$ in Γ, we discard $\mathcal{X}$. If $\mathcal{X}$ is a region-node, and $\mathcal{X}$ already occurs in P after the final window-node in P, we discard $\mathcal{X}$. If we do not discard $\mathcal{X}$, we construct a successor GCS-path P' by appending $\mathcal{X}$ to P, then compute its f-value as follows.

Suppose the first target-window in Γ unvisited by P' is $\Gamma[n]$. We optimize a trajectory τ_{a} with a collision-free portion $\tau_{\text{a},1}$ passing through the sets in P in sequence, followed by an obstacle-unaware portion $\tau_{\text{a},2}$ that travels to $\Gamma[n]$. We parameterize $\tau_{\text{a},1}$ with a line segment per set in P, and $\tau_{\text{a},2}$ with a single line segment. The objective of the trajectory optimization is a g-value plus an h-value. The g-value is the distance traveled by $\tau_{\text{a},1}$. The h-value is the distance traveled by $\tau_{\text{a},2}$, plus a term $h_n(t)$ which depends on the ending time t of $\tau_{\text{a},2}$. $h_n(t)$ lower bounds the cost of intercepting the remaining targets in Γ after departing $\Gamma[n]$ at time t, and we describe how to compute h_n in Section V-G.1.

The trajectory optimization is the same as the optimization performed for a GCS-path in FMC* [13], except for two differences. First, FMC* also optimizes an obstacle-unaware portion of the trajectory that departs $\Gamma[n]$ and intercepts all remaining targets, in place of our h_n value. Second, we constrain our computed trajectory to intercept $\Gamma[n]$ no later than a value $t_{\max,n}$, which is the latest time at which a feasible agent trajectory could depart $\Gamma[n]$, then intercept the remaining sequence of target-windows in Γ. We compute $t_{\max,n}$ for each $n \in \{1, 2, \ldots, \text{Len}(\Gamma)\}$ before beginning the search as follows. For $n = \text{Len}(\Gamma)$, we set $t_{\max,n} = \infty$. We then iterate backwards from $n = \text{Len}(\Gamma) - 1$ to $n = 1$, and $t_{\max,n} = \text{LFDT}(\Gamma[n], \Gamma[n+1], t_{\max,n+1})$.

Finally, if the trajectory optimization is infeasible, we discard P'. Otherwise, the trajectory optimization's optimal

[2]In practice, we check if $\vec{g}_{\text{lb}}[1] - \lambda < -10^{-4}$, as in [8]

cost is the f-value for P', and we push P' onto OPEN.

1) Constructing h_n Function: Consider the segment-graph $\mathcal{G}_{\text{seg}}$ for Γ, as defined in Section V-D. For a segment ξ in $\mathcal{G}_{\text{seg}}$, let $h_{\text{seg}}(\xi)$ be the cost of the shortest path in $\mathcal{G}_{\text{seg}}$ from ξ to ξ_0. Note that $h(\xi_0) = 0$, and for a segment ξ of $\Gamma[n]$ with $1 < n < \text{Len}(\Gamma)$, we have

$$h_{\text{seg}}(\xi) = \min_{\xi' \in \text{Segments}(\Gamma[n+1])} (c_{\text{seg}}(\xi, \xi') + h_{\text{seg}}(\xi')). \quad (11)$$

Before searching $\mathcal{G}_{\text{cs}}$, we compute h_{seg} for all ξ in $\mathcal{G}_{\text{seg}}$ as follows. We iterate backward from $n = \text{Len}(\Gamma) - 1$ to $n = 2$, and for each n, we compute the h-values for the segments of $\Gamma[n]$ using the h-values for $\Gamma[n+1]$ using (11).

Next, for each $n \in \{1, 2, \ldots, \text{Len}(\Gamma)\}$, we do the following. Let $\gamma_{i,j}$ be $\Gamma[n]$. We construct a matrix $A_n \in \mathbb{R}^{n_{\text{seg}}(\Gamma[n]) \times 2}$ whose kth row is $[\underline{t}_{i,j,k}, 1]$. We construct a vector $\vec{b}_n \in \mathbb{R}^{n_{\text{seg}}(\Gamma[n])}$ whose kth element is $h(\xi_{i,j,k})$. We then find the least-squares solution $\phi_n \in \mathbb{R}^2$ to the equation $A_n \phi_n = \vec{b}_n$. For a time $t \in [\underline{t}_{i,j}, \bar{t}_{i,j}]$, $\phi_n[1]t + \phi_n[2]$ approximates the minimum cost to intercept all remaining target-windows in Γ after departing $\Gamma[n]$ at time t. To adjust this into a lower bound, we construct another matrix B_n whose kth row is $[\bar{t}_{i,j,k}, 1]$, then compute a value $r_{\max}$ as the max element of the vector $\text{vertcat}(A_n, B_n)\phi_n - \text{vertcat}(\vec{b}_n, \vec{b}_n)$, where vertcat concatenates two matrices vertically. $r_{\max}$ is the largest overestimation of $h_{\text{seg}}(\xi_{i,j,k})$ that our approximation makes over all segment start and end times of $\gamma_{i,j}$. Then $h_n(t) = \phi_n[1]t + \phi_n[2] - r_{\max}$ lower-bounds the remaining cost of executing Γ after departing $\Gamma[n]$ at time t.

H. Feasible Solution Generation

To generate the initial incumbent, as well as a feasible set of tours $\mathcal{F}_{\text{new}}$ when the RSMP is infeasible, we extend the feasible solution generation from [8] to handle obstacles. This algorithm requires the EFAT and LFDT functions described previously. In [8], the values were computed using closed-form expressions, since [8] did not consider obstacles. Since we consider obstacles, we instead compute the values using the method from [14]. Otherwise, our initial feasible solution generation method is identical to BPRC's.

I. Caching EFAT and LFDT Values

Lazy BPRC computes EFAT and LFDT several times, possibly with the same arguments. We cache the values for each unique set of arguments to speed up the algorithm.

VI. THEORETICAL ANALYSIS

Lemma 1. *When we return no tours in the pricing problem,* (2) *is satisfied.*

Proof. Referring to values from the first paragraph of Section V-F, strong duality implies

$$\underline{c}(\theta) = \sum_{i=1}^{n_{\text{tar}}} \lambda_i + n_{\text{agt}} \lambda_0 \quad (12)$$

where the RHS is the cost of λ for the dual of the RSMP: this dual is the same as (18)-(21) in [11], but costs are replaced with lower bounds. If we return no tours, $c^*(\Gamma) - c_\lambda(\Gamma) \geq 0$ for all tours. This implies λ is feasible for the dual of LP-$\mathcal{B}$ (the same dual as (18)-(21) in [11]). The cost of λ for the dual of LP-$\mathcal{B}$ is $\sum_{i=1}^{n_{\text{tar}}} \lambda_i + n_{\text{agt}}\lambda_0$, which lower-bounds $c^*(\mathcal{B})$ by weak duality. Combining this with (12), we have (2). □

Theorem 1. *Lazy BPRC finds an optimal solution.*

Proof. Let $\mathcal{F}_{\text{opt}}$ be an optimal MT-VRP-O solution, let c_{opt} be its cost, and let θ_{opt} be the corresponding solution to ILP (1). We show by induction that whenever we execute Alg. 1, Line 5, either $\bar{c}_{\text{inc}} = c_{\text{opt}}$, or θ_{opt} is feasible for LP-$\mathcal{B}$ for some $\mathcal{B}$ on the stack.

Base Case The first node pushed onto the stack is $\mathcal{B} = \emptyset$, and LP-$\mathcal{B}$ is a relaxation of ILP (1), so θ_{opt} is feasible for LP-$\mathcal{B}$.

Induction Hypothesis Suppose on Line 5, either (i) $\bar{c}_{\text{inc}} = c_{\text{opt}}$, or (ii) θ_{opt} is feasible for LP-$\mathcal{B}$ for some $\mathcal{B}$ on the stack.

Induction Step Suppose (i) holds. We never increase $\bar{c}_{\text{inc}}$, and $\bar{c}_{\text{inc}}$ cannot become smaller than c_{opt} by the optimality of c_{opt}, so if Line 5 is ever executed again, (i) will still hold.

Now suppose (ii) holds. If $\mathcal{B}$ is not popped at this iteration, (ii) trivially holds at the next iteration. Next, suppose $\mathcal{B}$ is popped. Combining Lemma (1) with the feasibility of θ_{opt} for LP-$\mathcal{B}$, we have $\underline{c}(\theta) \leq c_{\text{opt}}$. Now we have two cases.

Case 1 Within the lazy evaluation loop, we set $\bar{c}_{\text{inc}} = c_{\text{opt}}$. Then (i) holds when we execute Line 5 next.

Case 2 $\bar{c}_{\text{inc}} \neq c_{\text{opt}}$ after the lazy evaluation loop. The optimality of c_{opt} then implies that $\bar{c}_{\text{inc}} > c_{\text{opt}}$. Combining this with $\underline{c}(\theta) \leq c_{\text{opt}}$, we have $\underline{c}(\theta) < \bar{c}_{\text{inc}}$. This means the condition on Line 15 fails and we attempt to generate successors for $\mathcal{B}$. No edge traversed by $\mathcal{F}_{\text{opt}}$ is in $\mathcal{B}$; if any edge traversed by $\mathcal{F}_{\text{opt}}$ were in $\mathcal{B}$, this would contradict (ii). Thus we have some edge to branch on when generating the successors $\mathcal{B}'$ and $\mathcal{B}''$. If we branch on an edge not traversed by $\mathcal{F}_{\text{opt}}$, then θ_{opt} is feasible for LP-$\mathcal{B}'$ and LP-$\mathcal{B}''$. If we branch on an edge e traversed by $\mathcal{F}_{\text{opt}}$, θ_{opt} is feasible for LP-$\mathcal{B}''$. Thus (ii) holds the next time we execute Line 5.

Thus, the induction hypothesis holds the next time we execute Line 5. Since the number of branch-and-bound nodes expanded in Alg. 1 cannot be larger than the finite number of subsets of $\mathcal{E}_{\text{tw}}$, Alg. 1 terminates. Termination only occurs when the stack becomes empty. This means at some point, we check Line 5, and the stack is empty, which means (ii) from the induction hypothesis cannot hold. Thus (i) holds at termination, implying that we found an optimal solution. □

VII. NUMERICAL RESULTS

We ran experiments on an Intel i9-9820X 3.3GHz CPU with 10 cores, hyperthreading disabled, and 128 GB RAM. We compared Lazy BPRC to two ablations. The first ablation, called "Non-Lazy BPRC," is the same algorithm, but whenever we generate a label l representing a tour Γ, and l is not currently dominated, we set $\vec{g}_{\text{lb}}[1] = \vec{g}_{\text{ub}}[1] = c^*(\Gamma)$; if Γ was unevaluated prior to this step, we evaluate Γ, update $\vec{g}_{\text{lb}}[1]$ and $\vec{g}_{\text{ub}}[1]$, then perform dominance checks again. The second ablation, called "No-Affine-Heuristic," replaces our heuristic in the SPP-GCS associated with tour

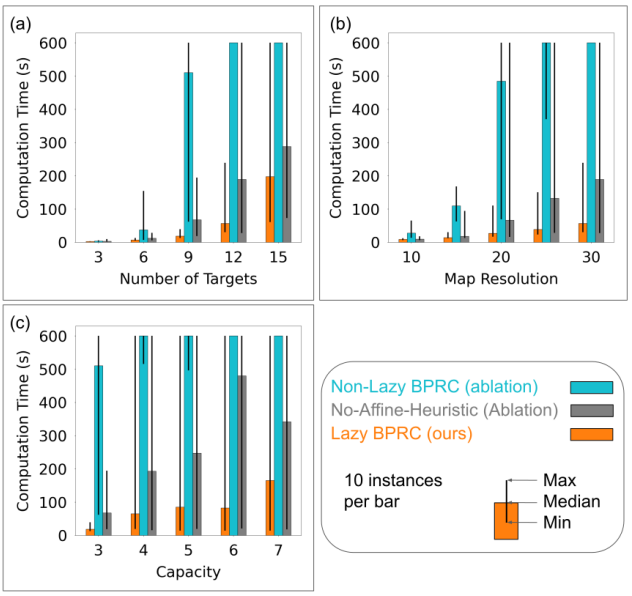

Fig. 3. (a) Varying the number of targets. Lazy BPRC shows smaller median runtime than the ablations, particularly for larger numbers of targets. (b) Varying the map resolution. Lazy BPRC's advantage in median runtime grows as we increase the map resolution. (c) Varying the capacity. Lazy BPRC has smaller median runtime than the ablations for all tested capacities.

Γ in Section V-G with the heuristic from FMC*. That is, when computing the f-value trajectory for a GCS-path P', the obstacle-unaware portion of the trajectory is required to intercept all target-windows in Γ unvisited by P', in sequence. Each algorithm parallelized successor generation in pricing and tour cost evaluation, the initial computation of pairwise distances between segments and segment-starts, and initial pairwise LFDT computations.

We generated problem instances by modifying the instance generation method from [14] to handle multiple agents. In every instance, each target had two time windows, demand 1, and speed within each time window generated uniformly at random between 0.5 and 1 m/s. Each instance had three agents with $v_{\max} = 4$ m/s. Our obstacle maps were square grids, but the agents and targets move in continuous space in the grids. We define the *map resolution* as the width of the obstacle map in grid cells. In our experiments, we varied the number of targets, map resolution, and capacity. We set the computation time limit to 10 min, per planner, per instance.

A. Experiment 1: Varying the Number of Targets

We fixed the map resolution to 30 and varied n_{tar} from 3 to 15, setting the capacity $d_{\max} = n_{\text{tar}}/n_{\text{agt}}$. Fig. 3 (a) shows the results. As n_{tar} increases, Lazy BPRC notably outperforms Non-Lazy BPRC in min, median, and max runtime, demonstrating that deferring the computation of tour costs is effective. Lazy BPRC also demonstrates smaller median and max runtimes than No-Affine-Heuristic, showing that our obstacle-aware heuristic leveraging continuity relaxation outperforms a heuristic that ignores obstacles.

B. Experiment 2: Varying the Map Resolution

We fixed n_{tar} to 12 and $d_{\max}$ to 4, then varied the map resolution from 10 to 30. Fig. 3 (b) shows the results. Lazy BPRC again demonstrates smaller median and max runtime than both ablations, and also smaller min runtime than Non-Lazy BPRC. Lazy BPRC's advantage grows with the map resolution because as we increase map resolution, the numbers of nodes and edges in the GCS tend to increase, making the GCS more expensive to search. Lazy BPRC outperforms Non-Lazy BPRC because it reduces the number of SPP-GCS queries, and Lazy BPRC outperforms No-Affine-Heuristic by speeding up each SPP-GCS query.

C. Experiment 3: Varying the Capacity

We fixed n_{tar} to 9 and the map resolution to 30, then varied the capacity $d_{\max}$ from 3 to 7. Fig. 3 (c) shows the results. Lazy BPRC shows smaller median runtime than both ablations, and also smaller min runtime than Non-Lazy BPRC. Lazy BPRC's max runtime hits the time limit for $d_{\max} \geq 4$.

Note that No-Affine-Heuristic's median runtime counter-intuitively drops when we increase $d_{\max}$ from 6 to 7. This occurs because in two instances, runtime became more than 2 times smaller when we increased $d_{\max}$; runtime did not change as significantly in the other 8 instances. The runtime dropped in these two instances for No-Affine-Heuristic because there were one or more tours whose evaluation required significant runtime for $d_{\max} = 6$, but simply never needed to be evaluated for $d_{\max} = 7$.

VIII. CONCLUSIONS

In this paper, we introduced Lazy BPRC, a new algorithm to find optimal solutions for the MT-VRP-O, and we demonstrated its benefits via ablation studies. One direction for future work is to pursue bounded-suboptimal solutions to enable scaling to more targets.

REFERENCES

[1] C. S. Helvig, G. Robins, and A. Zelikovsky, "The moving-target traveling salesman problem," *Journal of Algorithms*, vol. 49, no. 1, pp. 153–174, 2003.

[2] C. D. Smith, *Assessment of genetic algorithm based assignment strategies for unmanned systems using the multiple traveling salesman problem with moving targets*. University of Missouri-Kansas City, 2021.

[3] A. Stieber and A. Fügenschuh, "Dealing with time in the multiple traveling salespersons problem with moving targets," *Central European Journal of Operations Research*, vol. 30, no. 3, pp. 991–1017, 2022.

[4] J.-M. Bourjolly, O. Gurtuna, and A. Lyngvi, "On-orbit servicing: a time-dependent, moving-target traveling salesman problem," *International Transactions in Operational Research*, vol. 13, no. 5, pp. 461–481, 2006.

[5] B. Li, B. R. Page, J. Hoffman, B. Moridian, and N. Mahmoudian, "Rendezvous planning for multiple auvs with mobile charging stations in dynamic currents," *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 1653–1660, 2019.

[6] P. Toth and D. Vigo, *Vehicle routing: problems, methods, and applications*. SIAM, 2014.

[7] C. Archetti, L. Coelho, M. Speranza, and P. Vansteenwegen, "Beyond fifty years of vehicle routing: Insights into the history and the future," *European Journal of Operational Research*, 2025.

[8] A. Bhat, G. Gutow, Z. Ren, S. Rathinam, and H. Choset, "Optimal solutions for the moving target vehicle routing problem via branch-and-price with relaxed continuity," 2026. [Online]. Available: https://arxiv.org/abs/2603.00663

[9] M. Hammar and B. J. Nilsson, "Approximation results for kinetic variants of tsp," in *Automata, Languages and Programming: 26th International Colloquium, ICALP'99 Prague, Czech Republic, July 11–15, 1999 Proceedings 26*. Springer, 1999, pp. 392–401.

[10] L. Costa, C. Contardo, and G. Desaulniers, "Exact branch-price-and-cut algorithms for vehicle routing," *Transportation Science*, vol. 53, no. 4, pp. 946–985, 2019.

[11] D. Feillet, "A tutorial on column generation and branch-and-price for vehicle routing problems," *4or*, vol. 8, no. 4, pp. 407–424, 2010.

[12] T. Marcucci, J. Umenberger, P. Parrilo, and R. Tedrake, "Shortest paths in graphs of convex sets," *SIAM Journal on Optimization*, vol. 34, no. 1, pp. 507–532, 2024.

[13] A. Bhat, G. Gutow, B. Vundurthy, Z. Ren, S. Rathinam, and H. Choset, "A complete and bounded-suboptimal algorithm for a moving target traveling salesman problem with obstacles in 3d*," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 6132–6138.

[14] ——, "A complete algorithm for a moving target traveling salesman problem with obstacles," in *International Workshop on the Algorithmic Foundations of Robotics*. Springer, 2024.

[15] G. Ozbaygin, O. E. Karasan, M. Savelsbergh, and H. Yaman, "A branch-and-price algorithm for the vehicle routing problem with roaming delivery locations," *Transportation Research Part B: Methodological*, vol. 100, pp. 115–137, 2017.

[16] T. Asano, T. Asano, and H. Imai, "Shortest path between two simple polygons," *Information processing letters*, vol. 24, no. 5, pp. 285–288, 1987.

[17] M. Cui, D. D. Harabor, and A. Grastien, "Compromise-free pathfinding on a navigation mesh." in *IJCAI*, 2017, pp. 496–502.